Tandem Task Splitting Protocol

> Version: 0.1 (draft) Scope: SDK annotations, CLI build-time analysis, server dispatch rules, and node execution contract.

Table of contents

---

1. Annotation taxonomy

Annotations are how users express intent to the Tandem system. The CLI reads them at build time and uses them to drive compilation, validation, and manifest generation. There are two categories: **execution mode** annotations and **data** annotations.

1.1 Execution mode annotations

These control how and when a task runs.

Annotation	Alias	Meaning
`@tandem.compute`	—	One-shot function call. Input in, output out. Can be split.
`@tandem.split`	`@tandem.parallel`	Explicitly marks a function as splittable across nodes.
`@tandem.serve`	—	Long-lived request handler. Kept resident on a node.
`@tandem.async`	—	Fire-and-forget. Caller does not block for a result.
`@tandem.cron(expr)`	`@tandem.scheduled`	Triggered on a timer. Server wakes and assigns a node.
`@tandem.deferred`	—	Caller provides a callback or future; result delivered async.

Python examples:

```python
```

```
from tandem import task, compute, split, serve, async_task, cron, deferred

# One-shot computation
@tandem.compute
def generate_thumbnail(image: bytes, width: int, height: int) -> bytes:
    return resize(image, width, height)

# Explicitly splittable across nodes
@tandem.split(strategy="data_parallel", reducer="concat")
def encode_batch(videos: list[bytes]) -> list[bytes]:
    return [encode(v) for v in videos]

# Long-lived HTTP handler
@tandem.serve(port=8080)
def api_handler(request: Request) -> Response:
    return handle(request)

# Fire-and-forget
@tandem.async_task
def send_notification(user_id: str, message: str) -> None:
    push(user_id, message)

# Scheduled — runs every night at midnight UTC
@tandem.cron("0 0 * * *")
def nightly_report() -> None:
    generate_and_store_report()

# Deferred — caller receives a future
@tandem.deferred
def run_inference(prompt: str) -> str:
    return model.generate(prompt)
```

TypeScript examples:

```
typescript
```

```typescript
import { compute, split, serve, asyncTask, cron, deferred } from "@tandem/sdk";

// One-shot computation
export const generateThumbnail = compute(async (image: Buffer, width: number, h
  return resize(image, width, height);
});

// Splittable across nodes
export const encodeBatch = split(
  async (videos: Buffer[]): Promise<Buffer[]> => videos.map(encode),
  { strategy: "data_parallel", reducer: "concat" }
);

// Long-lived HTTP handler
export const apiHandler = serve(async (request: Request): Promise<Response> =>
  return handle(request);
}, { port: 8080 });

// Scheduled
export const nightlyReport = cron("0 0 * * *", async () => {
  await generateAndStoreReport();
});
```

1.2 Data annotations

These control how values are treated during validation and bundling.

Annotation	Meaning
`@tandem.immutable`	Value is bundled into the WASM artifact. Nodes receive a read-only copy.
`@tandem.constant`	Like `immutable`, but asserts the value never changes at runtime. Allows CLI-level optimizations.
`@tandem.param`	Explicit marker: this is per-invocation input data. Default assumption for all function parameters.
`@tandem.context`	Shared read-only config (region, credentials shape, feature flags) injected by the server at node startup.

Python examples:

```python
```

```python
# A large lookup table bundled into every node that runs classify()
LABEL_MAP: dict[int, str] = tandem.immutable({0: "cat", 1: "dog", 2: "bird"})

@tandem.split
def classify(images: list[bytes]) -> list[str]:
    # LABEL_MAP is valid here — it is immutable and bundled
    return [LABEL_MAP[predict(img)] for img in images]


# A compile-time constant — CLI can inline this
THRESHOLD: float = tandem.constant(0.85)

@tandem.compute
def filter_scores(scores: list[float]) -> list[float]:
    return [s for s in scores if s >= THRESHOLD]


# Context value — injected by the server, not the caller
@tandem.compute
def process(record: dict) -> dict:
    region = tandem.context("region")       # e.g. "us-east-1"
    env    = tandem.context("environment")  # e.g. "production"
    return transform(record, region=region, env=env)
```

---

2. Independence rule

A function is **independently executable** — and therefore eligible for distribution to a node — if and only if every value it reads satisfies one of the following conditions:

1. It is a **function parameter** (annotated `@tandem.param` or simply a normal argument).
2. It is annotated `@tandem.immutable` or `@tandem.constant` and has been bundled into the WASM artifact.
3. It is a `@tandem.context` value, injected by the server at node startup.
4. It is a **local variable** computed entirely from values that satisfy conditions 1–3.

If a function fails this rule, the CLI rejects it at build time with a descriptive error. The function will not be compiled to WASM.

2.1 Valid examples

python

```python
# ✓ All inputs are parameters
@tandem.split
def add_prefix(strings: list[str], prefix: str) -> list[str]:
    return [prefix + s for s in strings]


# ✓ Outer value is immutable — bundled into WASM
STOP_WORDS: set[str] = tandem.immutable({"the", "a", "an", "in"})

@tandem.split
def remove_stop_words(tokens: list[str]) -> list[str]:
    return [t for t in tokens if t not in STOP_WORDS]


# ✓ Local variable derived entirely from parameters
@tandem.compute
def normalize(values: list[float]) -> list[float]:
    total = sum(values)          # local, derived from param
    return [v / total for v in values]
```

2.2 Invalid examples — CLI will reject these

```python
```

```python
# × Reads a module-level mutable variable
counter = 0

@tandem.split
def increment_and_return(values: list[int]) -> list[int]:
    counter += 1          # ERROR: captures mutable outer scope
    return [v + counter for v in values]


# × Closure captures outer state
multiplier = 3

@tandem.split
def scale(values: list[float]) -> list[float]:
    return [v * multiplier for v in values]
    # ERROR: `multiplier` is not immutable, not a param, not context


# × Calls a non-independent helper
def fetch_config() -> dict:
    return requests.get("https://config-service/settings").json()

@tandem.compute
def process(record: dict) -> dict:
    config = fetch_config()    # ERROR: I/O side effect inside task
    return transform(record, config)
```

Correct version of the closure example:

```python
multiplier: float = tandem.immutable(3.0)

@tandem.split
def scale(values: list[float]) -> list[float]:
    return [v * multiplier for v in values]  # ✓
```

Correct version of the I/O example:

```python
```

```
@tandem.compute
def process(record: dict, config: dict) -> dict:
    # config is now a parameter — caller fetches it, passes it in
    return transform(record, config)
```

2.3 What the CLI checks

The CLI performs a static scope analysis pass before compilation. It rejects on:

- Module-level mutable variables referenced inside a task function
- Closures that capture outer scope state not marked `@tandem.immutable`
- Calls to functions that themselves fail the independence rule
- Unrouted I/O side effects (network calls, file reads not through a Tandem interface)
- Shared mutable objects (dicts, lists, class instances) passed by reference where mutation is possible

---

3. Task type classification

After annotations are read, the CLI assigns each task one of three **execution classes**. This class is written into the manifest and drives server-side dispatch.

3.1 Compute

Triggers: `@tandem.compute`, `@tandem.split`, `@tandem.parallel`

A pure function. A single invocation takes input and produces output. No persistent state is kept between calls. The server can fan it out to N nodes if a splitting strategy is set.

Use for: image processing, batch ML inference, data transformation, encoding, hashing, compression.

python

```
@tandem.split(strategy="data_parallel", reducer="concat")
def transcribe_audio(clips: list[bytes]) -> list[str]:
    return [whisper.transcribe(clip) for clip in clips]
```

3.2 Live / Hosted

Triggers: `@tandem.serve`, `@tandem.async`

Long-running. A node loads the WASM module and keeps it resident. The server routes incoming requests to it and load-balances across replicas. The server scales replica count

based on request volume.

Use for: HTTP handlers, WebSocket endpoints, event processors, streaming APIs.

```python
@tandem.serve(port=8080, replicas=3)
def chat_handler(request: Request) -> Response:
    session_id = request.headers["x-session-id"]
    history    = request.body["history"]
    reply      = model.respond(history)
    return Response(body={"reply": reply})
```

```python
# async variant — caller does not wait for result
@tandem.async_task
def process_upload(file_key: str, user_id: str) -> None:
    raw    = storage.get(file_key)
    result = pipeline(raw)
    storage.put(f"results/{user_id}/{file_key}", result)
```

3.3 Scheduled

Triggers: `@tandem.cron`, `@tandem.deferred`

Triggered by the server on a timer or event rather than by direct invocation. The server holds the schedule, wakes the task at the appropriate time, and assigns a cold node.

Use for: nightly report generation, queue draining, cache warming, periodic cleanup.

```python
@tandem.cron("0 6 * * 1")   # Every Monday at 6 AM UTC
def weekly_digest() -> None:
    users   = db.query("SELECT id FROM users WHERE digest_enabled = true")
    for uid in users:
        send_digest(uid)


@tandem.deferred(timeout_ms=60_000)
def run_long_inference(prompt: str, model_id: str) -> str:
    return models[model_id].generate(prompt)
```

When a `@tandem.split` task is invoked, the server selects a strategy from the manifest hint. The developer declares this in the annotation; the server may override it based on live conditions (node availability, queue depth, observed data size).

4.1 Data-parallel (default)

Input is sharded across N nodes. Each node processes its shard independently. Results are collected and merged by a reducer node or the server.

Best for: large batch inputs where each item can be processed independently.

```python
@tandem.split(
    strategy="data_parallel",
    reducer="concat",
    max_shards=32,
    min_shard_size=50
)
def embed_documents(docs: list[str]) -> list[list[float]]:
    return [embedding_model.encode(doc) for doc in docs]
```

The server shards `docs` into up to 32 batches of at least 50 items each, dispatches to 32 nodes, then concatenates the resulting embedding lists in order.

Built-in reducers:

Reducer	Behavior
`"concat"`	Appends result lists in shard order.
`"sum"`	Element-wise sum of numeric results.
`"merge"`	Dict merge (later shards win on key collision).
`"first"`	Returns the result of the first shard to complete.
`"custom"`	Points to a separate `@tandem.compute` function that receives all shard results.

Custom reducer example:

```python
```

```python
@tandem.compute
def average_reducer(shard_results: list[list[float]]) -> list[float]:
    n = len(shard_results)
    return [sum(col) / n for col in zip(*shard_results)]

@tandem.split(strategy="data_parallel", reducer=average_reducer)
def score_batch(inputs: list[dict]) -> list[float]:
    return [model.score(i) for i in inputs]
```

4.2 Pipeline

Output of one task feeds directly into the next. Each stage is a separate WASM module, potentially running on a different node. The server wires them together via a streaming interface.

Best for: multi-stage ETL, sequential processing where each stage is CPU-intensive.

python

```python
@tandem.pipeline(next="normalize_stage")
def ingest_stage(raw: bytes) -> dict:
    return parse(raw)

@tandem.pipeline(next="embed_stage")
def normalize_stage(record: dict) -> dict:
    return clean(record)

@tandem.pipeline(next="store_stage")
def embed_stage(record: dict) -> dict:
    record["embedding"] = model.encode(record["text"])
    return record

@tandem.compute
def store_stage(record: dict) -> str:
    return db.insert(record)
```

The manifest wires these stages together. The server can run `ingest_stage` on node A and `embed_stage` on a GPU-equipped node B without the developer managing that routing.

4.3 Replicated

N identical copies run on N nodes simultaneously. Used for live tasks where fault tolerance and low latency matter. The server routes each incoming request to the least-loaded replica.

```python
@tandem.serve(replicas=5, strategy="replicated")
def inference_server(request: Request) -> Response:
    result = model.predict(request.body["input"])
    return Response(body={"result": result})
```

The server spins up 5 replicas. If one node becomes unhealthy, the server redirects traffic to the remaining 4 and schedules a replacement.

4.4 Single

No splitting. Runs on exactly one node. Used when a task maintains state, holds a persistent connection, or requires strict ordering.

```python
@tandem.serve(strategy="single")
def order_processor(event: dict) -> dict:
    # Must process events in strict order — no parallelism
    return apply_event_to_state(event)
```

---

5. Immutable bundling pass

When the CLI encounters a `@tandem.immutable` or `@tandem.constant` value referenced inside a task, it runs a bundling pass to decide how the value reaches the node.

5.1 Compile-time constants

If the value is a literal or can be evaluated at build time, the CLI embeds it directly into the WASM module's initial memory.

```python
# Embedded directly into the WASM artifact
SUPPORTED_FORMATS: frozenset[str] = tandem.constant(frozenset({"jpg", "png", "w

@tandem.compute
def validate_format(filename: str) -> bool:
    ext = filename.rsplit(".", 1)[-1].lower()
    return ext in SUPPORTED_FORMATS
```

5.2 Runtime immutables

If the value is not known until runtime (e.g., loaded from a file or fetched at startup), the CLI records it in the manifest as a **bundle requirement**. The server resolves and delivers it to the node as part of the invocation payload. Nodes receive it as a sealed argument and cannot mutate it.

python

```python
# Loaded at server startup, delivered to each node as a sealed bundle
MODEL_WEIGHTS: bytes = tandem.immutable(load_weights("model_v3.bin"))

@tandem.split(strategy="data_parallel")
def run_inference(inputs: list[dict]) -> list[float]:
    model = deserialize(MODEL_WEIGHTS)
    return [model.predict(i) for i in inputs]
```

Manifest entry produced by the CLI:

json

```json
{
  "immutable_bundles": [
    {
      "name": "MODEL_WEIGHTS",
      "source": "load_weights(\"model_v3.bin\")",
      "resolve": "server_startup",
      "delivery": "invocation_payload"
    }
  ]
}
```

5.3 Context values

Context values are not bundled into the artifact. Instead, they are injected by the server at node startup from a per-deployment config. They are available to any task running on that node.

python

```
@tandem.compute
def tag_record(record: dict) -> dict:
    region = tandem.context("region")
    record["source_region"] = region
    return record
```

Context values are declared in a deployment config file, not in user code:

json

```
{
  "context": {
    "region": "us-east-1",
    "environment": "production",
    "log_level": "warn"
  }
}
```

---

6. Manifest split hints

The `.tandem` artifact's `manifest.json` carries a `split_hints` block. The CLI writes this at build time; the server reads it at dispatch time and may override values based on live conditions.

6.1 Full manifest example

json

```
{
  "name": "video-pipeline",
  "version": "1.2.0",
  "tasks": [
    {
      "name": "encode_batch",
      "wasm": "tasks/encode_batch.wasm",
      "execution_class": "compute",
      "split": {
        "strategy": "data_parallel",
        "max_shards": 64,
        "min_shard_size": 10,
        "reducer": "concat",
        "timeout_per_shard_ms": 15000,
        "retry_on_shard_failure": true,
        "max_retries_per_shard": 2
      },
      "immutable_bundles": ["CODEC_TABLE"],
      "memory_mb": 512,
      "timeout_ms": 120000
    },
    {
      "name": "transcoding_server",
      "wasm": "tasks/transcoding_server.wasm",
      "execution_class": "serve",
      "split": {
        "strategy": "replicated",
        "replicas": 4,
        "scale_policy": "request_rate",
        "scale_up_threshold": 100,
        "scale_down_threshold": 10
      },
      "memory_mb": 1024,
      "timeout_ms": 0
    },
    {
      "name": "nightly_cleanup",
      "wasm": "tasks/nightly_cleanup.wasm",
      "execution_class": "scheduled",
      "schedule": {
        "cron": "0 2 * * *",
        "timezone": "UTC",
        "allow_overlap": false
```

```
      },
      "memory_mb": 256,
      "timeout_ms": 300000
    }
  ],
  "graph": {
    "pipeline_stages": [
      {"from": "ingest_stage", "to": "normalize_stage"},
      {"from": "normalize_stage", "to": "embed_stage"},
      {"from": "embed_stage", "to": "store_stage"}
    ]
  }
}
```

6.2 Split hint fields

Field	Type	Default	Description
`strategy`	string	`"single"`	`data_parallel`, `pipeline`, `replicated`, `single`
`max_shards`	int	`16`	Maximum number of parallel shards for data-parallel tasks.
`min_shard_size`	int	`1`	Minimum number of items per shard. Server will not split below this.
`reducer`	string or ref	`"concat"`	How shard results are combined.
`timeout_per_shard_ms`	int	`30000`	Per-shard timeout. If exceeded, shard is retried or failed.
`retry_on_shard_failure`	bool	`true`	Whether to retry a failed shard on a different node.
`max_retries_per_shard`	int	`1`	Maximum retry attempts per shard before marking it failed.
`replicas`	int	`1`	For replicated/serve tasks: target replica count.
`scale_policy`	string	`"static"`	`static`, `request_rate`, `queue_depth`, `cpu`

7. Server dispatch rules

The server is logic-free with respect to task content — it never parses or executes task code. It only reads `manifest.json` and routes accordingly.

7.1 Compute task dispatch

```
1. Receive job (task name + input payload).
2. Read manifest: execution_class == "compute".
3. If input size > (min_shard_size × 2) AND max_shards > 1:
     a. Shard input according to strategy.
     b. Dispatch each shard to a healthy node with available capacity.
     c. Attach immutable bundles to each shard payload.
     d. Collect shard results as they arrive.
     e. Run reducer (on a dedicated reducer node or server-side if reducer is
built-in).
     f. Return merged result to caller.
4. If input is too small to shard:
     a. Dispatch entire payload to a single node.
     b. Return result directly.
```

7.2 Live / Hosted task dispatch

```
1. At deploy time: spin up `replicas` nodes, each loading the WASM module.
2. Keep a health registry of live replica nodes.
3. On each incoming request:
     a. Select least-loaded healthy replica (round-robin or weighted).
     b. Forward request payload to that node.
     c. Stream or return response to caller.
4. If a node becomes unhealthy:
     a. Remove from registry.
     b. Spawn a replacement node.
     c. Re-route in-flight requests if possible; otherwise return 503.
5. If scale_policy is dynamic:
     a. Monitor the chosen metric (request rate, queue depth, CPU).
     b. Scale up by spawning new replicas when threshold is crossed.
     c. Scale down by draining and terminating replicas when below threshold.
```

```
1. At deploy time: register the cron expression in the schedule queue.
2. At each trigger time:
     a. If allow_overlap == false and a previous run is still active, skip.
     b. Otherwise, assign a cold node.
     c. Deliver the task with any context values.
     d. Log completion or failure; do not retry unless explicitly configured.
```

7.4 Shard failure and retry

```
1. If a shard times out or the node fails:
     a. If retry_on_shard_failure == true AND retries <
max_retries_per_shard:
          - Re-dispatch the same shard payload to a different healthy node.
     b. If retries exhausted:
          - Mark the whole job as failed.
          - Return error to caller with shard index and reason.
```

---

8. Tandem objects

A **TandemObject** is a named, immutable, content-addressed value stored on the server. Rather than passing large data payloads through the invocation path, a caller uploads the data once and references it by ID. All tasks that receive the ID can read the object; none can mutate it.

This is the recommended pattern for large datasets, model weights, media files, and other inputs shared across many task invocations.

8.1 Usage

python

```python
# Upload once, reference many times
object_id = tandem.objects.put(large_dataset)   # returns a content-addressed I

# Pass the ID as a task parameter — not the data itself
results = encode_batch.invoke(input_ref=object_id)
```

python

```python
@tandem.split(strategy="data_parallel")
def encode_batch(input_ref: tandem.ObjectRef) -> list[bytes]:
    # The node resolves the ref from the object store
    data = tandem.objects.get(input_ref)
    return [encode(item) for item in data]
```

8.2 Object lifecycle

Action	Behavior
`tandem.objects.put(value)`	Stores value, returns a content-addressed `ObjectRef`.
`tandem.objects.get(ref)`	Resolves ref on the node. Value is cached locally if already fetched.
`tandem.objects.delete(ref)`	Marks object for deletion. In-flight tasks using it complete first.
`tandem.objects.ttl(ref, seconds)`	Sets an expiry. Object is deleted after TTL unless refreshed.

Objects are immutable after creation. Any modification requires creating a new object with a new ref.

---

9. Async result delivery

Tasks annotated `@tandem.async` or `@tandem.deferred` do not return results inline. Instead, they are stored in the result store and delivered via one of the following mechanisms.

9.1 Polling

The simplest approach. The caller receives a job ID and polls for the result.

```python
```

```python
# Caller side
job = run_long_inference.invoke_async(prompt="Explain gravity", model_id="gpt-x
print(job.id)   # "job_a3f9c2"

# Later...
result = tandem.jobs.get(job.id)
if result.status == "complete":
    print(result.value)
elif result.status == "failed":
    print(result.error)
```

9.2 Webhook

The caller registers a URL. The server POSTs the result when the task completes.

python

```python
job = run_long_inference.invoke_async(
    prompt="Explain gravity",
    model_id="gpt-x",
    on_complete="https://myapp.com/webhooks/tandem"
)
```

Server delivers:

json

```json
{
  "job_id": "job_a3f9c2",
  "status": "complete",
  "value": "Gravity is a force that...",
  "duration_ms": 4821
}
```

9.3 Future / promise (SDK-level)

The SDK wraps the polling loop in a future for ergonomic use in async runtimes.

python

```python
# Python
async def main():
    future = await run_long_inference.invoke_future(prompt="Explain gravity", m
    result = await future   # blocks until complete
    print(result)
```

```typescript
// TypeScript
const result = await runLongInference({ prompt: "Explain gravity", modelId: "gp
console.log(result);
```

9.4 Job result retention

Results are retained in the result store for 24 hours by default. This is configurable per task:

```python
@tandem.deferred(result_ttl_seconds=3600)   # retain for 1 hour
def run_inference(prompt: str) -> str:
    return model.generate(prompt)
```

---

10. Error handling and retries

10.1 Task-level errors

If a task function raises an exception, the node captures it and reports it to the server as a structured error. The server propagates it to the caller.

```python
@tandem.compute
def parse_record(raw: bytes) -> dict:
    try:
        return json.loads(raw)
    except json.JSONDecodeError as e:
        raise tandem.TaskError(f"Invalid JSON: {e}", retryable=False)
```

`retryable=True` signals the server that the error is transient and the task should be retried on another node. `retryable=False` (the default) means the error is deterministic and retrying would produce the same result — the server fails fast.

10.2 Shard-level errors in data-parallel tasks

When a shard fails:

- If `retry_on_shard_failure: true` in the manifest, the server re-dispatches the shard to a different node.
- After `max_retries_per_shard` attempts, the shard is marked permanently failed.
- The entire job is then marked failed, and the caller receives an error that includes the failing shard index, so partial results can be recovered if needed.

10.3 Node failure

If a node dies mid-execution:

- The server detects the failure via the health heartbeat.
- For compute tasks: the shard is automatically re-dispatched. This is safe because independence is guaranteed — there is no state to recover.
- For serve tasks: the replica is removed from the pool and a replacement is started. In-flight requests are returned as 503 to the caller, who should retry.
- For scheduled tasks: the run is logged as failed. No automatic retry unless configured.

---

11. Open questions

These design decisions are unresolved and should be settled before CLI implementation begins.

Reducer ownership Who executes the reducer for data-parallel tasks — a dedicated reducer node, the server itself, or the calling client? The recommended answer is a dedicated reducer node (another `@tandem.compute` task), which keeps the server logic-free. However, built-in reducers (`concat`, `sum`, `merge`) could be handled server-side to avoid an extra hop.

Partial split functions Can a function contain both splittable and non-splittable sections? The current recommendation is no: the developer should decompose into one `@tandem.split` function for the parallelizable part and a separate `@tandem.compute` for the serial part. This keeps the independence contract clean.

TandemObject consistency If a TandemObject is used as an `@tandem.immutable` bundle and is also passed as a parameter via `ObjectRef`, which resolution path takes priority? These should probably be unified into one mechanism.

Pipeline backpressure If stage B in a pipeline is slower than stage A, the server needs a backpressure mechanism to prevent stage A from flooding the queue. A configurable

queue depth per stage edge is the likely answer, but the specific semantics are not yet defined.

Async result retention on node When a `@tandem.deferred` task completes on a node, does the node hold the result until the server retrieves it, or does it push immediately? The push model is simpler; the pull model gives the node more control over memory pressure.